Order-derivatives of all degrees: $\partial_\lambda^k S_\lambda(a) = \int_0^\infty (\log t)^k t^{\lambda-1} e^{-\beta t+\beta a\sqrt{t}}\,dt$

Claude, with Daniel Murfet

2026-09-06

Abstract

Unit 13 of the grammar-paper §4 autoformalisation: the general-k form of `eq:log_lambda_derivative`, $\partial_\lambda^k S_\lambda(a) = \int_0^\infty (\log t)^k t^{\lambda-1} e^{-\beta t+\beta a\sqrt{t}}\,dt$. The k-th derivative in the order parameter inserts $(\log t)^k$ under the integral; this is the analytic backbone of the paper's log-insertion identities (`lem:log_shift`, `prop:LogODE`, `lem:log_generating`), which tie SLT pole multiplicities to order-derivatives. Proved by induction on k on top of the $k = 1$ machinery. Zero `sorry`/`axiom`.

1 Setting

Unit 11 established $k = 1$ ($\partial_\lambda S_\lambda = \int (\log t) t^{\lambda-1} e^{\cdots}$). Iterating requires two things at every degree: the $|\log t|^k$-weighted integrand must be integrable (the dominating function grows one log per differentiation), and each differentiation step must be justified under the integral sign.

2 The things formalised

```
theorem abs_log_pow_le (t  : ) (m : ) (ht : 0 < t) (h  : 0 <  ) :
    |Real.log t| ^ m   (m /  ) ^ m * (t ^   + t ^ (- ))

theorem log_pow_fluctuation_integrableOn (  c a : ) (m : ) (h  : 0 <  ) (hc : 0 < c) :
    IntegrableOn (fun t => |Real.log t|^m * t^(c-1) * Real.exp (-
*t +  *a*Real.sqrt t)) (Set.Ioi 0)

theorem hasDerivAt_logpow_integral (  lam a :  ) (k :  ) (h  : 0 <  ) (hlam : 0 < lam) :
    HasDerivAt (fun l =>   t in Set.Ioi 0, (Real.log t)^k * t^(l-1) * Real.exp (...))
      (  t in Set.Ioi 0, (Real.log t)^(k+1) * t^(lam-1) * Real.exp (...)) lam

theorem iteratedDeriv_fluctuation_order (  a :  ) (h  : 0 <  ) (k :  ) :
     lam, 0 < lam → iteratedDeriv k (fun l => fluctuation  l a) lam
      =  t in Set.Ioi 0, (Real.log t)^k * t^(lam-1) * Real.exp (...)
```

3 Proof strategy

Power bound. Raising the $k = 1$ bound $|\log t| \le (t^\varepsilon + t^{-\varepsilon})/\varepsilon$ to the m-th power is done per regime: for $t \ge 1$, $\log t \le (m/\delta) t^{\delta/m}$ so $(\log t)^m \le (m/\delta)^m t^\delta$ (`pow_le_pow_left` then $(t^{\delta/m})^m = t^\delta$);

symmetrically for $t < 1$ via t^{-1}. This keeps the exponent at $\pm\delta$ (rather than a binomial spread) and reduces to two $t^q e^{-bt}$ integrals with $q = c - 1 \pm c/2 > -1$.

Generic step. `hasDerivAt_logpow_integral` differentiates the $(\log t)^k$ integral in λ exactly as the $k = 1$ case, with the pointwise derivative $(\log t)^k \cdot (t^{l-1} \log t) = (\log t)^{k+1} t^{l-1}$ (`pow_succ`) and the dominating function the $|\log t|^{k+1}$-weighted two-power bound.

Induction. `iteratedDeriv_succ` peels one derivative; the integral representation and the k-th iterated derivative agree on the *open* set $(0, \infty)$ (induction hypothesis), so they are eventually equal near any $\lambda > 0$, and `Filter.EventuallyEq.deriv_eq` transfers the derivative to g_k, which `hasDerivAt_logpow_integral` evaluates. The base case is $(\log t)^0 = 1$.

4 Roadblocks and resolutions

The whole file compiled on the first build — the $k = 1$ unit's playbook transferred cleanly. The three points of care:

- The integrand's norm factorises as $|\log t|^k \cdot t^{l-1} \cdot e^{\cdots}$ via `abs_mul`+`abs_pow`+`abs_of_pos` (the polynomial and exponential factors are positive), never `abs_of_nonneg` on the whole (signed) product.
- The induction needs a *neighbourhood* equality, not a pointwise one, to differentiate; since the two functions agree on all of $(0, \infty)$ — an open set containing λ — `Filter.eventuallyEq_of_mem (Ioi_mem_nhds hlam)` supplies it.
- Keeping the two dominating exponents in the form $\pm(c/2) + (c - 1)$ (as `Real.rpow_add` produces them) avoids any exponent-normalisation rewrite.

5 What was Mathlib, what was new

Mathlib supplied `Real.log_le_rpow_div`, `pow_le_pow_left` , the parametric-FTC lemma, `iteratedDeriv_succ`/`_zero`, and `Filter.EventuallyEq.deriv_eq`. New: the $|\log t|^m$ power bound, the $|\log t|^m$-weighted fluctuation integrability (all m), the generic order-differentiation step, and the general-k log-insertion identity itself.

6 Lessons

An iterated parameter-derivative identity is cleanest as a *generic single step* ($\int(\log)^k \to \int(\log)^{k+1}$) plus an `iteratedDeriv` induction, with the step's dominating function carrying one extra log; the induction's differentiation is licensed by open-set (hence neighbourhood) agreement of the closed form with the iterated derivative, transferred by `EventuallyEq.deriv_eq`.

7 Follow-ups

With $\partial_\lambda^k S$ available as log insertions, `lem:log_shift` ($\int t^{\nu-1}(c - \log t)^p e^{\cdots} = (c - \partial_\nu)^p S_\nu$, binomial theorem), `prop:LogODE` (differentiate the Weber ODE k times in λ), and `lem:log_generating` ($\sum \frac{z^p}{p!}(c - \partial_\nu)^p S_\nu = e^{zc} S_{\nu - z}$, Taylor's theorem in the order) are the natural next targets. The ladder-operator commutator $[b, b^\dagger] = 1$ still wants a Weyl-algebra framework.